

Relatório de Testes Unitários

Sistema de Gestão de Consultas — Núcleo da Pessoa Idosa (NPI)

Dupla 2 · Sprint 1 · maio de 2026

1. Resumo Executivo

Este relatório documenta a implementação da base de testes unitários para o sistema de gestão de consultas do NPI, desenvolvida pela Dupla 2 durante a Sprint 1. O trabalho seguiu a regra estabelecida no workshop: nenhuma funcionalidade considerada pronta sem ao menos um teste cobrindo seu comportamento.

Testes implementados	20
Arquivos criados	4
Edge cases cobertos	12
Branch	feature/unit-tests

2. Contexto do Projeto

O Sistema de Gestão de Consultas foi desenvolvido para a recepção do Núcleo da Pessoa Idosa (NPI), com foco no fluxo diário de agendamento de consultas para pacientes idosos. O usuário principal é a recepcionista, que utiliza o sistema para marcar, confirmar e acompanhar as consultas dos pacientes.

Os dados centrais manipulados pelo sistema são:

- Nome do paciente e do responsável
- Telefone do paciente e do responsável
- Data e horário da consulta (formato DD/MM/YYYY e HH:MM)
- Setor de atendimento: Geriatria, Fisioterapia, Assistência Social, entre outros

A responsabilidade da Dupla 2 foi criar a camada de funções puras — validators, formatters e calculators — que sustentam as regras de negócio do sistema sem depender de banco de dados, rede ou qualquer efeito colateral externo.

3. O Que Foi Implementado

3.1 validators.js — Funções de Validação

Arquivo responsável por validar os dados de entrada antes de qualquer processamento. Todas as funções retornam boolean e não lançam exceções.

`isValidNome(nome)`

- Valida o nome do paciente ou responsável
- Rejeita null, undefined e qualquer valor que não seja string
- Exige mínimo de 3 caracteres
- Aceita apenas letras (incluindo acentos) e espaços — sem números ou símbolos

`isValidTelefone(telefone)`

- Valida o telefone do paciente ou responsável
- Aceita formatos com ou sem máscara: (81) 99999-9999 ou 81999999999
- Rejeita null e strings com menos de 10 ou mais de 11 dígitos numéricos

`isValidDataConsulta(data)`

- Valida a data informada para o agendamento
- Aceita exclusivamente o formato brasileiro DD/MM/YYYY
- Rejeita datas no passado — só permite agendar a partir de hoje
- Rejeita datas inexistentes, como mês 13 ou dia 32

`isValidHorario(horario)`

- Valida o horário da consulta
- Aceita apenas o formato HH:MM com dois pontos
- Respeita o expediente do NPI: 08:00 às 18:00
- Rejeita 18:01 ou qualquer horário após o fechamento

3.2 formatters.js — Funções de Formatação

Arquivo responsável por formatar e normalizar dados para exibição e armazenamento. As funções lançam `TypeError` quando recebem entradas inválidas.

`formatarData(dataInput)`

- Recebe data no formato DD/MM/YYYY e retorna no mesmo formato para exibição
- Lança `TypeError` para null, undefined ou string em formato incorreto
- Usada no painel de consultas e nos lembretes de impressão

`normalizarNome(nome)`

- Remove espaços extras no início e no fim do nome
- Capitaliza a primeira letra de cada palavra: 'maria silva' → 'Maria Silva'
- Lança `TypeError` para null ou qualquer valor não-string
- Garante padronização dos nomes antes de salvar no banco

3.3 calculators.js — Regras de Negócio Puras

Arquivo responsável pelos cálculos e regras de negócio que não dependem de banco ou servidor.

`diasParaConsulta(dataConsulta)`

- Calcula quantos dias faltam para a data da consulta a partir de hoje
- Retorna 0 para consulta no dia, positivo para consultas futuras e negativo para passadas
- Lança `TypeError` para null ou string que não representa uma data válida

`isHorarioDisponivel(horario, agendamentosDoDia)`

- Verifica se um horário já está ocupado na lista de agendamentos do dia
- Retorna false se o horário já existe, true se está livre
- Lança `TypeError` se a lista de agendamentos não for um array
- Evita conflitos de horário ao criar um novo agendamento

4. Testes Unitários Implementados

Os testes foram escritos com Vitest e organizados por função usando `describe/it`. Cada teste cobre um comportamento específico, com foco em quatro categorias:

valido	edge case	limite	exception
entrada correta aceita	null / undefined	valor no limite da regra	deve lancar <code>TypeError</code> / <code>RangeError</code>

#	Funcao	Teste	Descricao	Tipo
01	<code>isValidNome</code>	aceita nome valido	Verifica que 'Maria Silva' passa — letras, espacos, acima de 3 chars.	valido
02	<code>isValidNome</code>	rejeita null	null nao causa crash e retorna false, protegendo campos nao preenchidos.	edge case
03	<code>isValidNome</code>	rejeita nome com numeros	'Maria2' deve falhar — numeros nao sao permitidos em nomes.	limite
04	<code>isValidTelefone</code>	aceita celular com 11 digitos	(81) 99999-9999 tem 11 digitos — valido para celular brasileiro.	valido
05	<code>isValidTelefone</code>	rejeita telefone curto	'1234' tem 4 digitos — abaixo do minimo de 10. Deve retornar false.	limite
06	<code>isValidTelefone</code>	rejeita null	Campo de telefone vazio nao pode passar pela validacao.	edge case
07	<code>isValidDataConsulta</code>	rejeita data no passado	'01/01/2020' esta no passado — nao pode ser usada para novo agendamento.	limite
08	<code>isValidDataConsulta</code>	rejeita formato ISO	'2025-12-25' esta em formato ISO — sistema so aceita DD/MM/YYYY.	edge case
09	<code>isValidDataConsulta</code>	rejeita null	Campo de data vazio nao pode criar um agendamento.	edge case

10	<code>isValidHorario</code>	aceita horario dentro do expediente	'09:30' esta dentro do expediente do NPI (08:00-18:00).	valido
11	<code>isValidHorario</code>	rejeita 18:01	Expediente fecha as 18:00. Um minuto apos ja esta fora do limite.	limite
12	<code>isValidHorario</code>	rejeita null	Horario nao preenchido nao pode criar agendamento sem horario.	edge case
13	<code>formatarData</code>	retorna data no formato brasileiro	'25/12/2025' e retornada como '25/12/2025' para exibicao no painel.	valido
14	<code>formatarData</code>	lanca TypeError para null	null deve lancar TypeError — funcao nao aceita entrada invalida.	exception
15	<code>normalizarNome</code>	capitaliza e remove espacos extras	' maria silva ' vira 'Maria Silva' antes de salvar no banco.	valido
16	<code>normalizarNome</code>	lanca TypeError para null	null deve lancar TypeError — protege chamada com dado inexistente.	exception
17	<code>diasParaConsulta</code>	lanca TypeError para null	null deve lancar TypeError — calculo de dias exige data valida.	exception
18	<code>isHorarioDisponivel</code>	retorna false para horario ocupado	'09:00' ja existe na lista — retorna false, evitando conflito.	limite
19	<code>isHorarioDisponivel</code>	retorna true para horario livre	'10:00' nao esta na lista — horario disponivel, pode agendar.	valido
20	<code>isHorarioDisponivel</code>	lanca TypeError se lista for null	Lista de agendamentos deve ser array — null lanca TypeError.	exception

5. Estrutura de Arquivos Criados

Os seguintes arquivos foram criados na branch feature/unit-tests:

Arquivo	Descricao
<code>backend/src/utils/validators.js</code>	Funcoes puras de validacao de dados de entrada
<code>backend/src/utils/formatters.js</code>	Funcoes puras de formatacao e normalizacao de dados
<code>backend/src/utils/calculators.js</code>	Regras de negocio puras: disponibilidade e calculo de dias
<code>backend/src/utils/__tests__/utils.test.js</code>	Suite de testes com 20 casos unitarios usando Vitest

6. Futuras Implementacoes

As funcionalidades abaixo serao implementadas em proximas sprints, baseadas diretamente nas historias do backlog do sistema NPI. Cada item ja tem testes planejados seguindo a regra de cobertura obrigatoria.

6.1 isValidSetor — Validacao de Setor

Backlog: Historia 6 — Filtro de Consultas por Setor e Especialidade

A recepcionista precisa filtrar consultas por setor (Geriatria, Fisioterapia, Assistencia Social, Nutricao, Psicologia). Para isso, o sistema precisa validar se o setor informado e um dos cadastrados no NPI antes de salvar ou filtrar.

Testes planejados:

- Validar que 'geriatria', 'fisioterapia' e 'assistencia social' sao aceitos
- Validar que setores nao cadastrados como 'cardiologia' sao rejeitados
- Garantir que a validacao e case-insensitive (GERIATRIA = geriatria)
- Rejeitar null e string vazia sem lancar execucao

6.2 formatarLembreteImpressao — Formatacao do Lembrete Fisico

Backlog: Historia 3 — Emissao de Lembrete de Consulta para Impressao

Ao marcar o retorno de um paciente, a recepcionista precisa gerar um lembrete fisico para entregar ao idoso ou responsavel. A funcao deve receber o agendamento e retornar um objeto formatado com todos os campos prontos para impressao.

Testes planejados:

- Formatar objeto com paciente (capitalizado), data (DD/MM/YYYY), horario e setor (maiúsculas)
- Gerar texto formatado: 'Lembrete de Consulta / Paciente: X / Data: X / Horario: X / Setor: X'
- Lancar TypeError se faltar qualquer campo obrigatorio (paciente, data, horario, setor)
- Lancar TypeError para null ou argumento que nao seja objeto

6.3 isConsultaAtrasada — Deteccao de Atraso

Backlog: Historia 10 — Destaque de Consultas Atrasadas ou Proximas

A recepcionista precisa identificar rapidamente consultas que ja passaram do horario marcado para agir com agilidade. A funcao recebe data e horario da consulta e verifica se o momento ja passou comparando com o horario atual.

Testes planejados:

- Retornar true para consulta com data e horario no passado
- Retornar false para consulta futura
- Lancar TypeError se data ou horario forem null ou undefined
- Funcionar corretamente no limite (consulta marcada para o minuto atual)

6.4 contarPorSetor — Contagem por Setor

Backlog: Historia 6 — Filtro de Consultas por Setor e Especialidade

O sistema deve exibir o total de agendamentos por setor no topo da lista filtrada. A funcao recebe a lista completa de agendamentos e retorna um objeto com a contagem agrupada por setor.

Testes planejados:

- Contar corretamente: [{setor: 'geriatria'}, {setor: 'geriatria'}] retorna {geriatria: 2}
- Retornar objeto vazio para lista de agendamentos vazia
- Lançar TypeError se o argumento não for um array
- Ignorar agendamentos que não possuem campo setor definido

6.5 gerarMensagemWhatsApp — Mensagem de Confirmação

Backlog: Historia 2 — Sistema de Confirmação Assistida via WhatsApp

A recepcionista precisa que o sistema prepare automaticamente a mensagem de confirmação para o responsável, de forma que ela apenas clique em enviar no WhatsApp Web. A funcao monta o texto com os dados do agendamento no formato exato definido no backlog.

Testes planejados:

- Gerar mensagem no formato: 'Ola [Responsavel], confirmamos a consulta de [Paciente] para o dia [Data] as [Hora]. Podemos confirmar?'
- Lançar TypeError se qualquer campo obrigatório estiver ausente ou nulo
- Formatar a data automaticamente para DD/MM/YYYY dentro da mensagem
- Aceitar nome do responsável e do paciente já normalizados

7. Conclusão

A base de testes unitários foi implementada com sucesso, cobrindo os quatro pilares exigidos: entradas inválidas, null/undefined, exceções e limites de regras de negócio. Os 20 testes implementados garantem que as funções puras do sistema se comportam corretamente em cenários normais e em casos extremos, servindo como base sólida para as próximas sprints.

As cinco funcionalidades listadas como futuras implementações estão diretamente mapeadas ao backlog do produto e representam o próximo passo natural para ampliar a cobertura de testes do sistema NPI.